Lanturn: Measuring Cryptoeconomic Smart-Contract Security through Learning

Kushal Babel

Cornell Tech

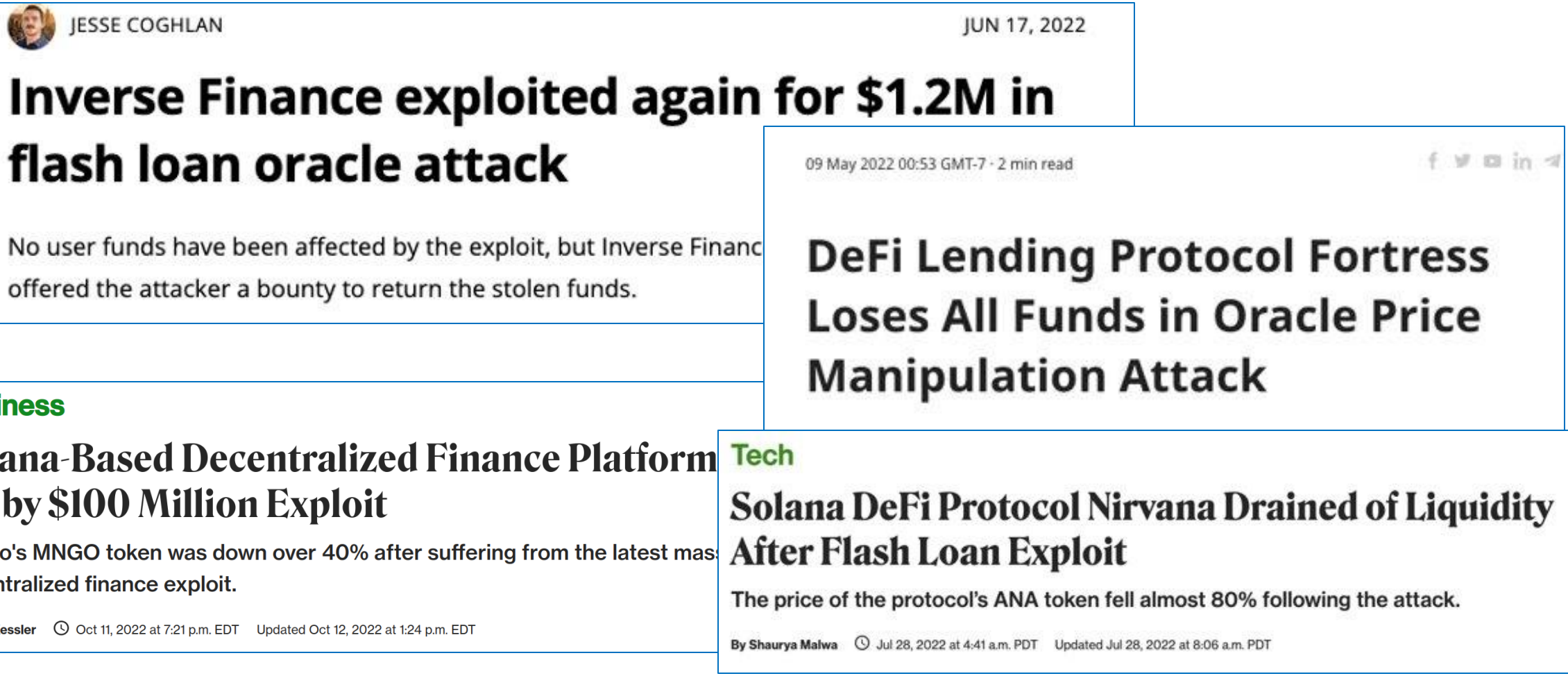
IC3 Retreat '23

with Mojan Javaheripi, Mahimna Kelkar, Farinaz Koushanfar, Ari Juels

Decentralized Finance (DeFi)

- DeFi applications have grown in popularity, handling billions of dollars
- This is powered by the rise of smart-contracts

The Perils



JESSE COGHLAN JUN 17, 2022

Inverse Finance exploited again for $1.2M in flash loan oracle attack

No user funds have been affected by the exploit, but Inverse Financ offered the attacker a bounty to return the stolen funds.

09 May 2022 00:53 GMT-7 · 2 min read

DeFi Lending Protocol Fortress Loses All Funds in Oracle Price Manipulation Attack

Business

Solana-Based Decentralized Finance Platform Hit by $100 Million Exploit

Mango's MNGO token was down over 40% after suffering from the latest mas decentralized finance exploit.

By Sam Kessler Oct 11, 2022 at 7:21 p.m. EDT Updated Oct 12, 2022 at 1:24 p.m. EDT

Tech

Solana DeFi Protocol Nirvana Drained of Liquidity After Flash Loan Exploit

The price of the protocol's ANA token fell almost 80% following the attack.

By Shaurya Malwa Jul 28, 2022 at 4:41 a.m. PDT Updated Jul 28, 2022 at 8:06 a.m. PDT

Complex Interactions

- Smart-Contracts can interoperate very easily.

- Modular Smart-Contracts (Money Legos) interact with each other to enable richer applications.

- Examples:
  1. Lending contract can use Decentralized Exchange(DEX) contract as price oracle.
  2. Multiple DEX contracts can be aggregated together.
  3. FlashLoans + DEX
  4. FlashLoans + Lending Contract ...

Talk Outline

- MEV, an example
- Previous approaches
- Lanturn
  - Overview of the system
  - Optimization Module
  - Simulation Module
- Evaluation
- Conclusion

MEV, An Example

Swap 100 USD into ETH

USER

ETH price goes up as a result

MEV, An Example

Swap 10,000 USD into ETH

Swap 100 USD into ETH

Swap ETH into 10,000 USD

MINER

USER

MINER

ETH price ↑

MINER executes a "*sandwich*" attack and profits off the user.

MEV, Informally...

MEV = Maximal Extractable Value (or Miner Extractable Value)

Ability of miners/validators/bots to extract value by reordering, inserting or censoring transactions

***EV* =** Value extracted in a given transaction sequence

MEV = max(*EV* for *any* transaction sequence)

We can use MEV as the measure of Cryptoeconomic Security, as it models the worst case adverserial advantage.

Prior Approaches: Heuristics Based

- Most works encode a specific attack strategy
- These approaches do not generalizable to new contracts or new transaction types
- As a result, these approaches do not attempt to find the worst-case adversarial advantage.

- L. Zhou, K. Qin, C. F. Torres, D. V. Le and A. Gervais, "High-Frequency Trading on Decentralized On-Chain Exchanges," 2021 IEEE Symposium on Security and Privacy (SP), 2021, pp. 428-445, doi: 10.1109/SP40001.2021.00027.

Prior Approaches: Clockwork Finance

- Given a set of user transactions, and templates for transaction insertions by the miner, find the optimal value of the inserted transactions and optimal order of transactions that maximize MEV.
- Formal guarantees on the optimality of obtained MEV.
- Not scalable for complex contracts such as contracts with *loops* (Curve Finance), or large number of transactions (>10)

K. Babel, P. Daian, M. Kelkar and A. Juels, "Clockwork Finance : Automated Analysis of Economic Security in Smart Contracts" 2023 IEEE Symposium on Security and Privacy (SP), To Appear.

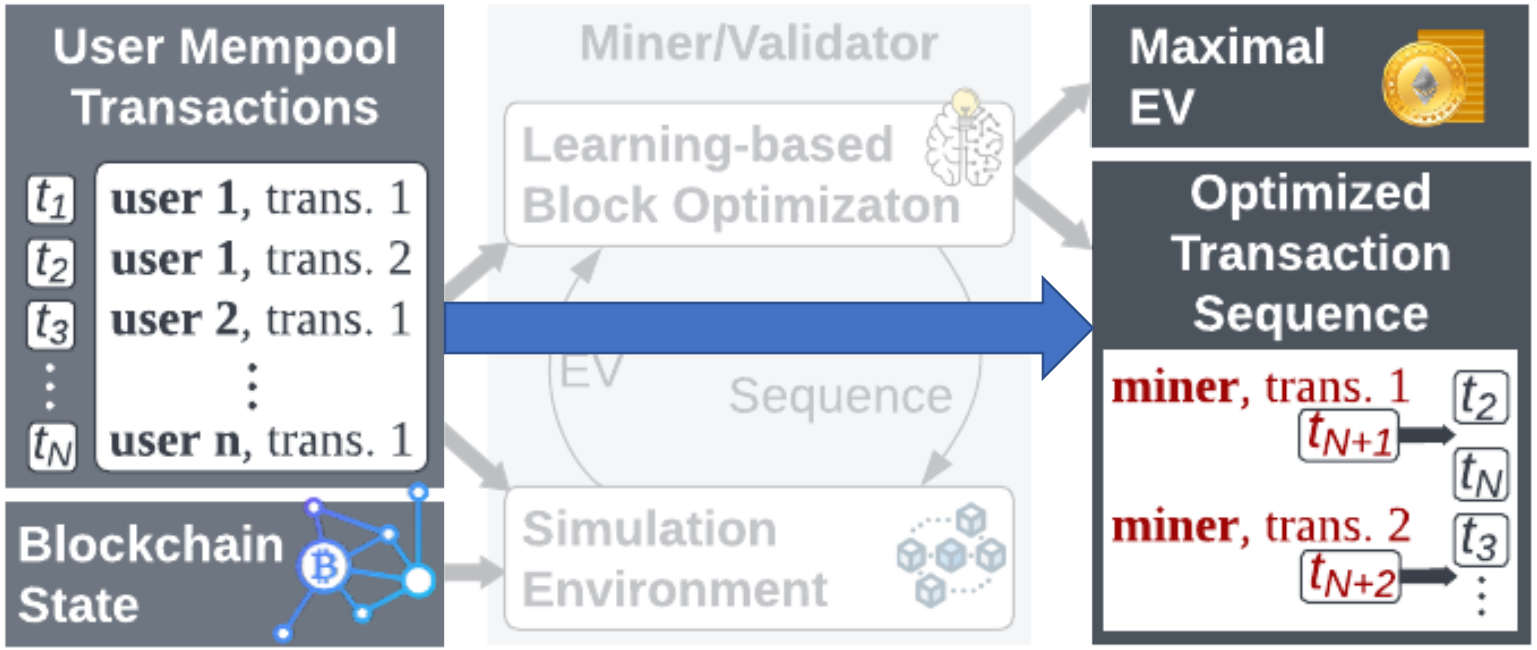
This Work: Lanturn

Contributions:

- **Learning based** problem formulation and simulation framework.
- **Native Smart-Contract Execution** : Generalize to *any* smart-contract by treating smart-contract bytecode as black box simulation environment
- **Scalability** : Uncover MEV strategies spanning large (>30) number of transactions efficiently, and without modelling (complex) smart-contract behavior.
- **Near-optimality** of uncovered MEV, better than MEV exploited in the wild

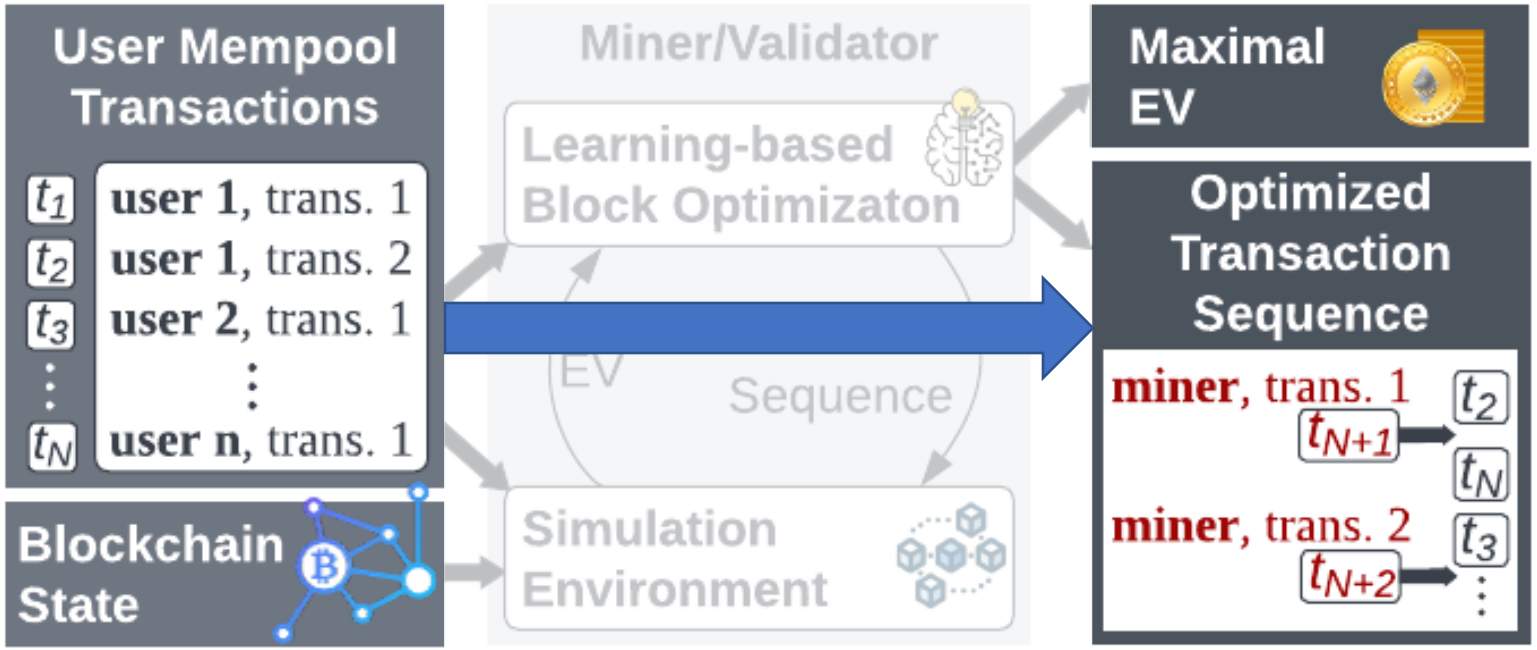
System Overview

- Given a pool of user transactions and the current blockchain state, Lanturn *automatically* learns to maximize the miner's EV.



System Overview

- The strategies include:
  - How to order the transactions in the block?
  - What transactions can the miner insert to take advantage of the block?



System Overview

- The strategies include:
  - How to order the transactions in the block?
  - What transactions can the miner insert to take advantage of the block?

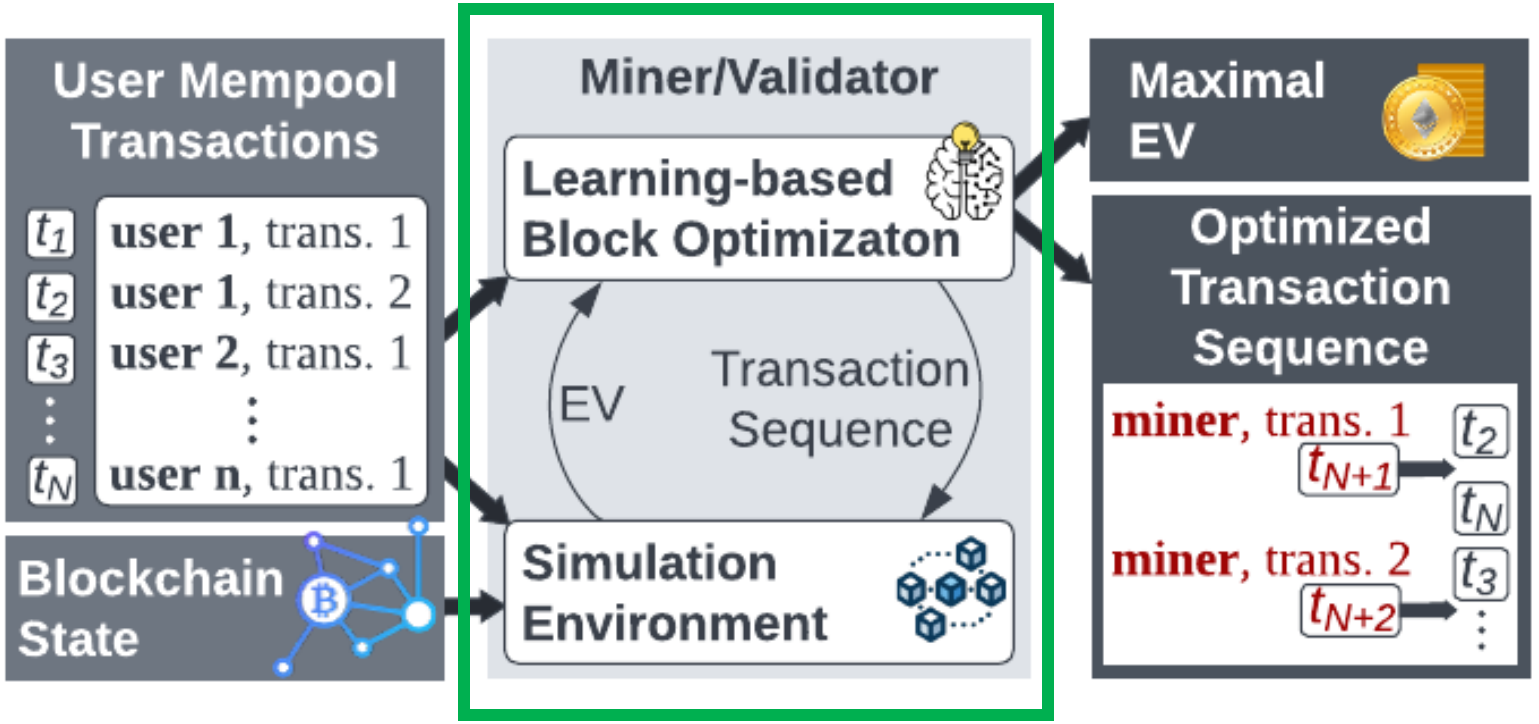
- For the latter question, the miner transactions are selected from a set of templates like:

$$\textbf{swap}\ \ \alpha_0 \times \texttt{token}_0\ \ \textbf{with}\ \ \alpha_1 \times \texttt{token}_1$$

In this example,
- The tokens are filled with those that user transactions have interacted with.
- The alpha values are found by Lanturn such that they maximize EV.

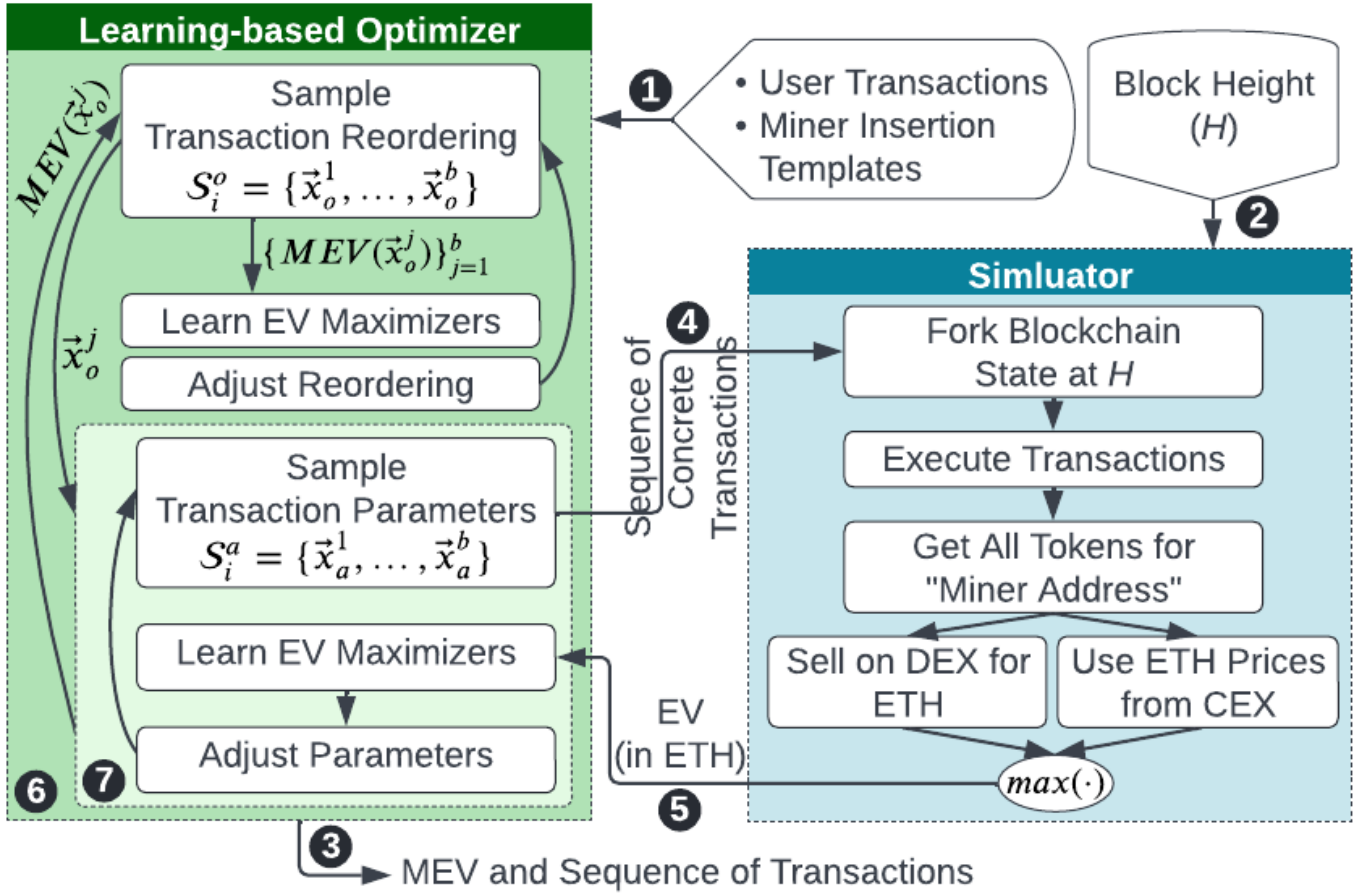
System Overview

- Learning is done through iterative interactions between our optimization module and simulator.



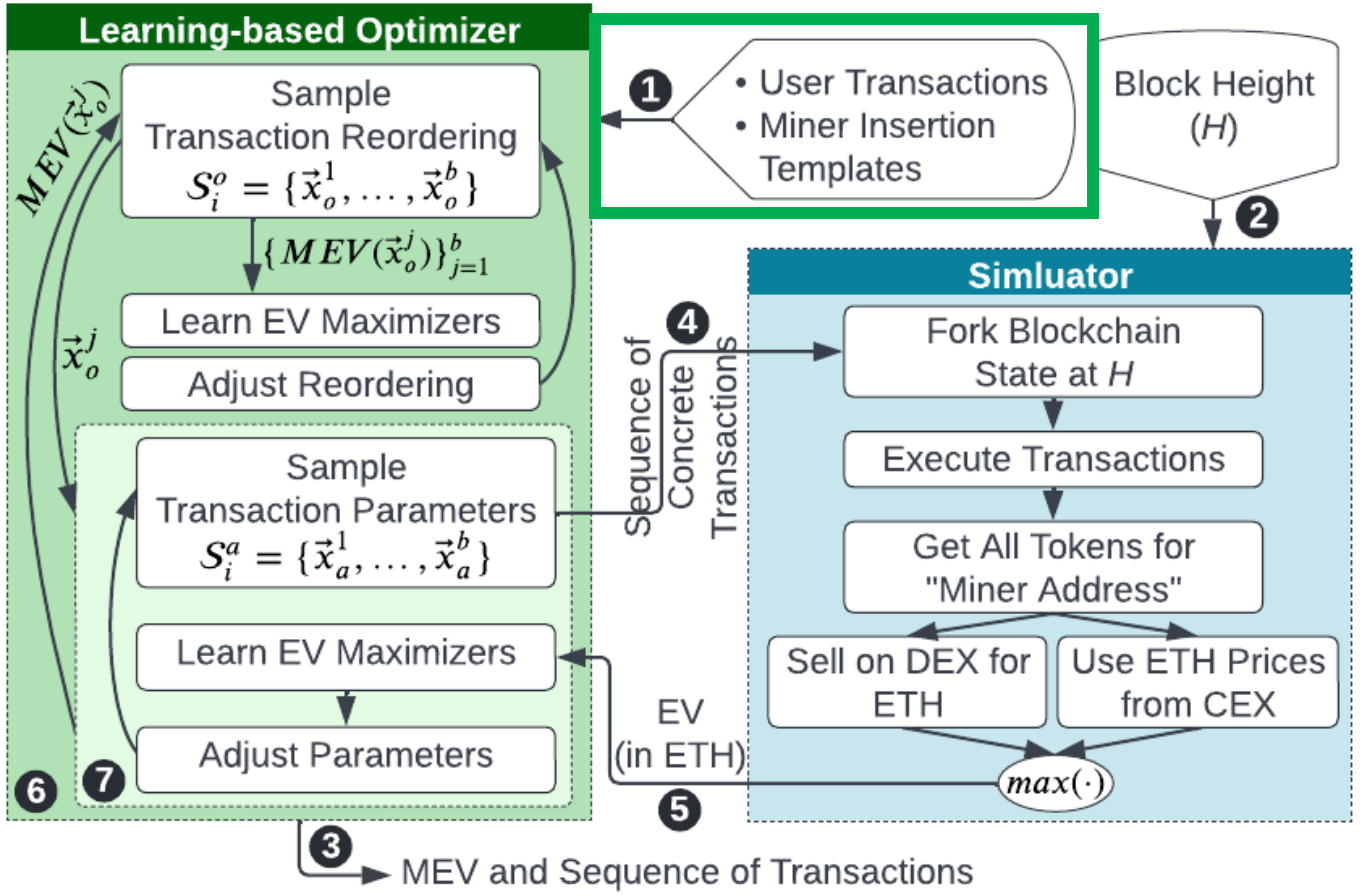
Deeper Look Inside

- High-level view of the internals of Lanturn components:



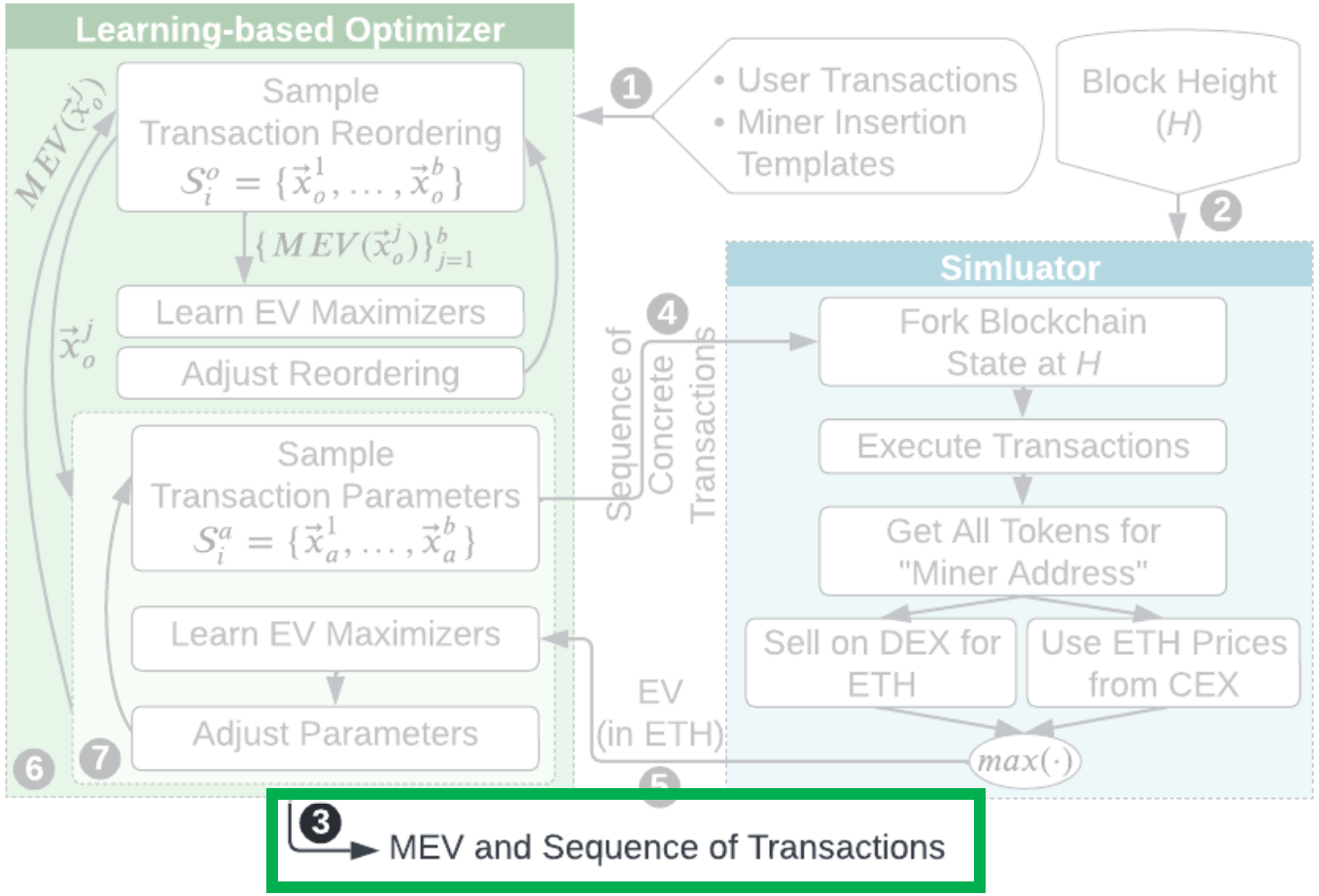
Deeper Look Inside

- Inputs to the optimizer (❶):
  - User transactions to be mined
  - Templates for miner-inserted transactions



Deeper Look Inside

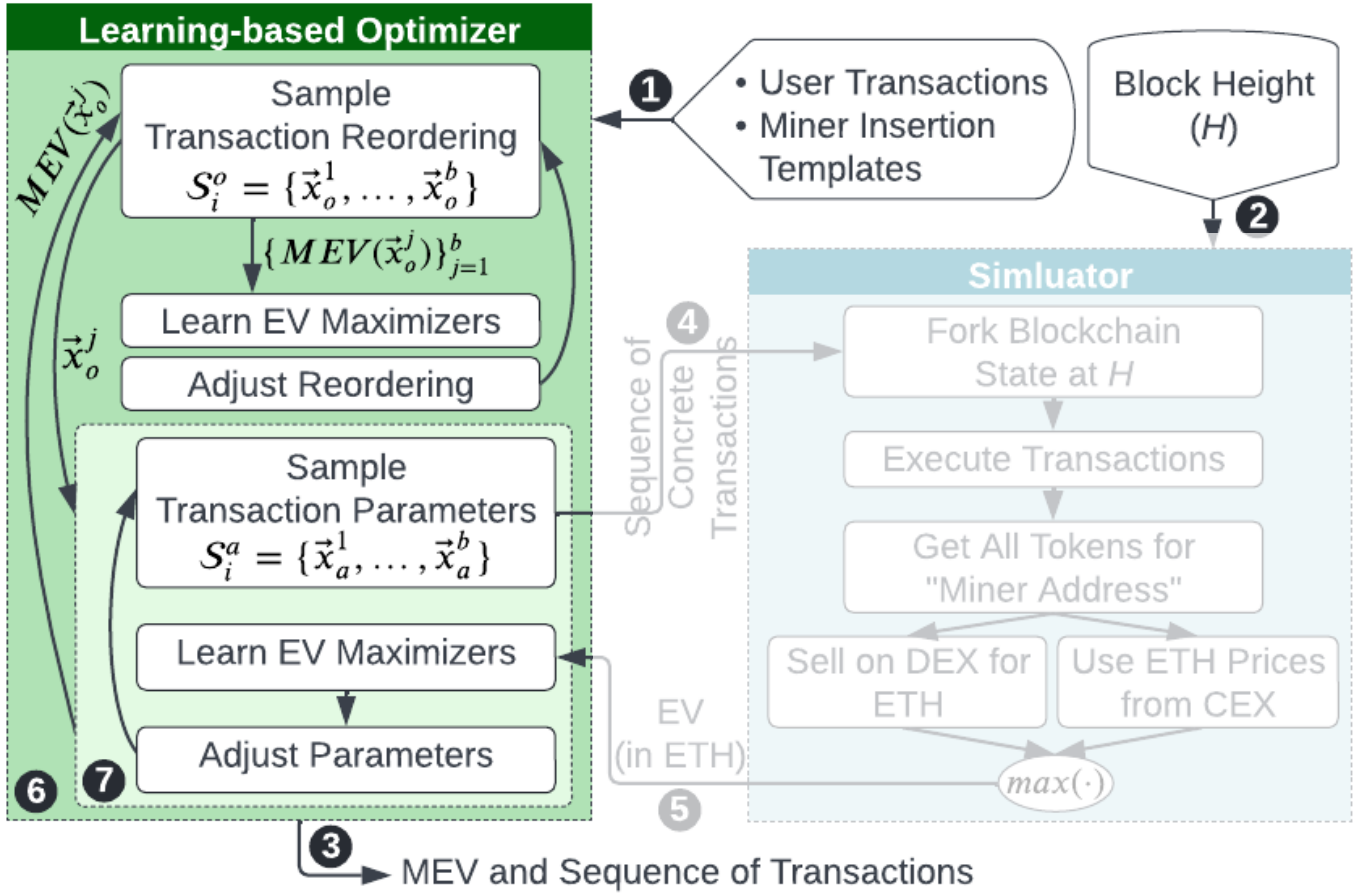
- Output of the optimizer (❸):
  - Maximal EV
  - Optimal transaction order



Deeper Look Inside

- Optimization has two hierarchical learning loops:

1. The outer loop learns the optimal order of transactions (❻).
2. The inner loop learns the optimal transaction amounts for the miner (❼).



Lanturn Learning-based Optimizer

- Problem Formulation: $\max_{\vec{x} \in X} F(\vec{x})$

1. Objective function (F): Miner's EV in terms of Eth
2. Design Variables ($\vec{x}$): Order of transactions (x_o), transaction amounts (x_a)

Bi-level optimization:

$$\max_{\vec{x_o} \in X_o} F(\vec{x_o})$$

Find the best transaction order

$$\text{where } F(\vec{x_o}) = \max_{\vec{x_a} \in X_a} f(\vec{x_o}, \vec{x_a})$$

Find the best transaction amounts

3. Optimization Bounds (X_o, X_a): Accepted range of values for each design variable, a.k.a, the search space

Lanturn Learning-based Optimizer

Goal: empirically search for optimal hyperparameter vector:

$$\vec{x}^* = \underset{\vec{x} \in X}{\operatorname{argmax}} F(\vec{x})$$

What does optimality mean?

- High EV for the miner

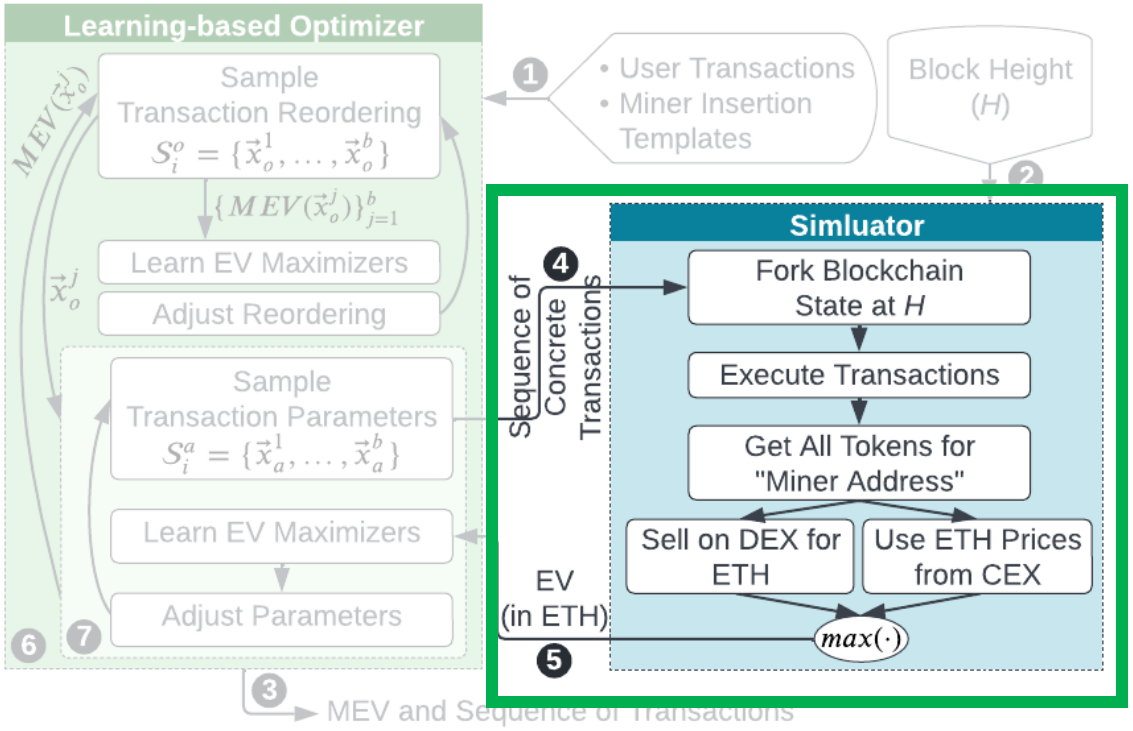
Lanturn Learning-based Optimizer

Goal: empirically search for optimal hyperparameter vector:

$$\vec{x}^* = \underset{\vec{x} \in X}{\operatorname{argmax}}\ F(\vec{x})$$

What does empirical mean?

- No analytical solution
- Relies on accurate simulations of EV



Lanturn Learning-based Optimizer

Goal: empirically search for optimal hyperparameter vector:

$$\vec{x}^* = \underset{\vec{x} \in X}{\operatorname{argmax}}\ F(\vec{x})$$

Search-space is not small, even for the below toy problem with only 3 transactions, the search space covers 48000 possibilities.

How to search the large space?
-Fast convergence
-Computational efficiency
-Parallelizable

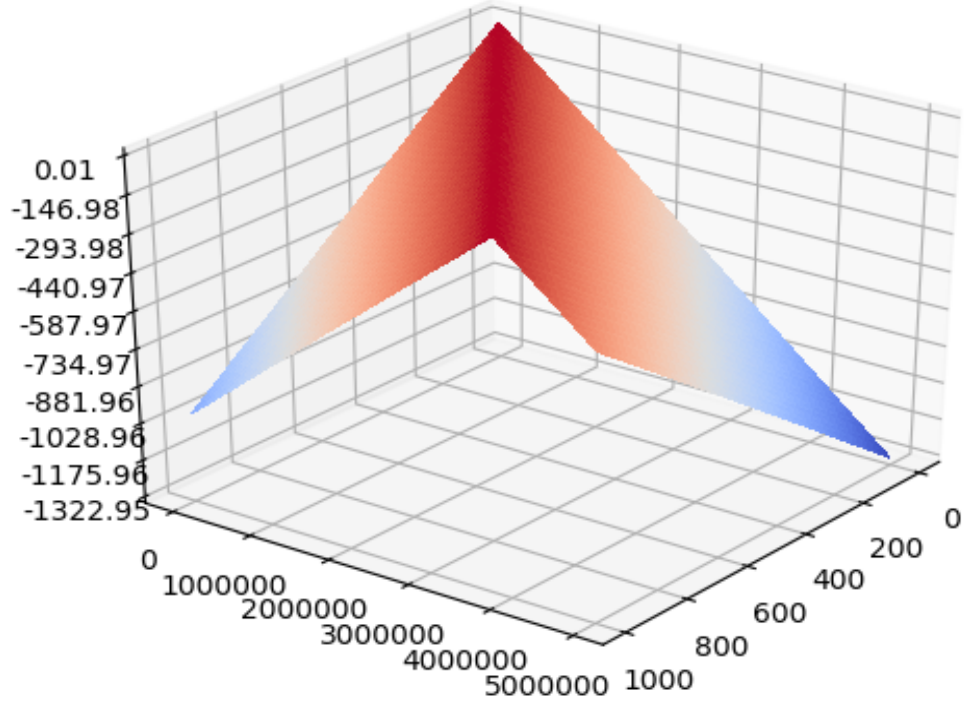
Transactions:

- User adds 10 `eth` and 40000 `usdc` of liquidity
- Miner swaps for `usdc` by providing α_0 `eth`
- Miner swaps for `eth` by providing α_1 `usdc`

Allowed amounts: $\alpha_0 \in [1, 5],\ \ \alpha_1 \in [4000, 20000]$

Search-space Geometry

- Visualization for finding transaction amounts for an example problem with miner executing a "Just-in-time" liquidity strategy.
- **Search goal:** Identify the peak region.
- Challenges: In general -
  - Non-monotonic
  - Non-convex
  - High correlations between variables
  - High-dimensional space



- horizontal plane: alpha hyperparameters
- vertical axis: objective function (EV)

Efficient Search Requirements

- To ensure fast exploration:

Evaluate members of a collection $\{\vec{x}\}_{i=1}^{N}$ in parallel

A good collection has many members in the neighborhood of:

$$F(\vec{x}_n) \approx \max_{\vec{x}} F(\vec{x})$$

- Optimization based on adaptive sampling:
  - Adjust the sampling distributions by learning from previous evaluations

Optimization through Adaptive Sampling

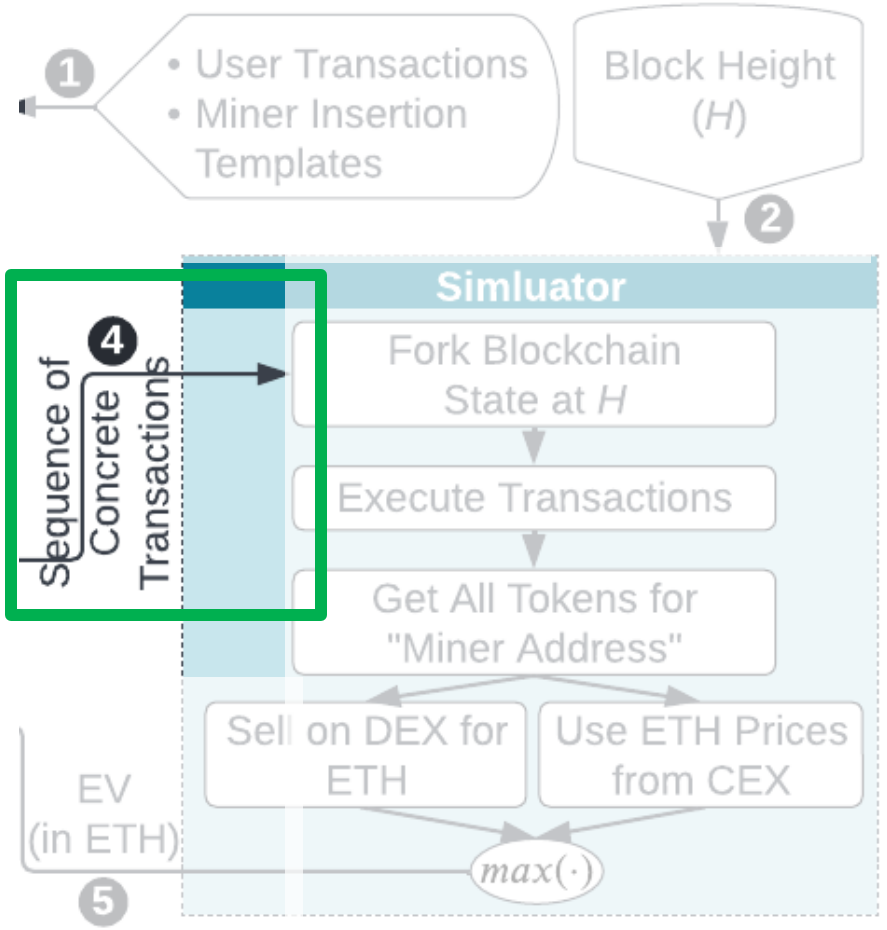
- Pool of Samples: $\mathcal{S} = \{\vec{x}\}_{i=1}^{N}$ with current maximizer $\vec{x}_{\mathcal{S}}^{*}$
- To maximize the probability of finding a near-optimal solution:

$$\max_{\mathcal{S}} \mathcal{P}\left(F(\vec{x}_{\mathcal{S}}^{*}) > \gamma F^{*}\right) \qquad \text{s.t. } 0 < \gamma < 1$$

- Choose samples sequentially: $\mathcal{S} = \mathcal{S}_1 \cup \mathcal{S}_2 \cup \ldots \cup \mathcal{S}_T$
- $F\left(\mathcal{S}_{[1:t]}\right)$ is used in choosing $\mathcal{S}_{t+1}$
- $\mathcal{S}_i$ s are random sets : $\mathcal{S}_{t+1} \sim \mathcal{G}_{t+1}(.|\mathcal{S}_{[1:t]})$

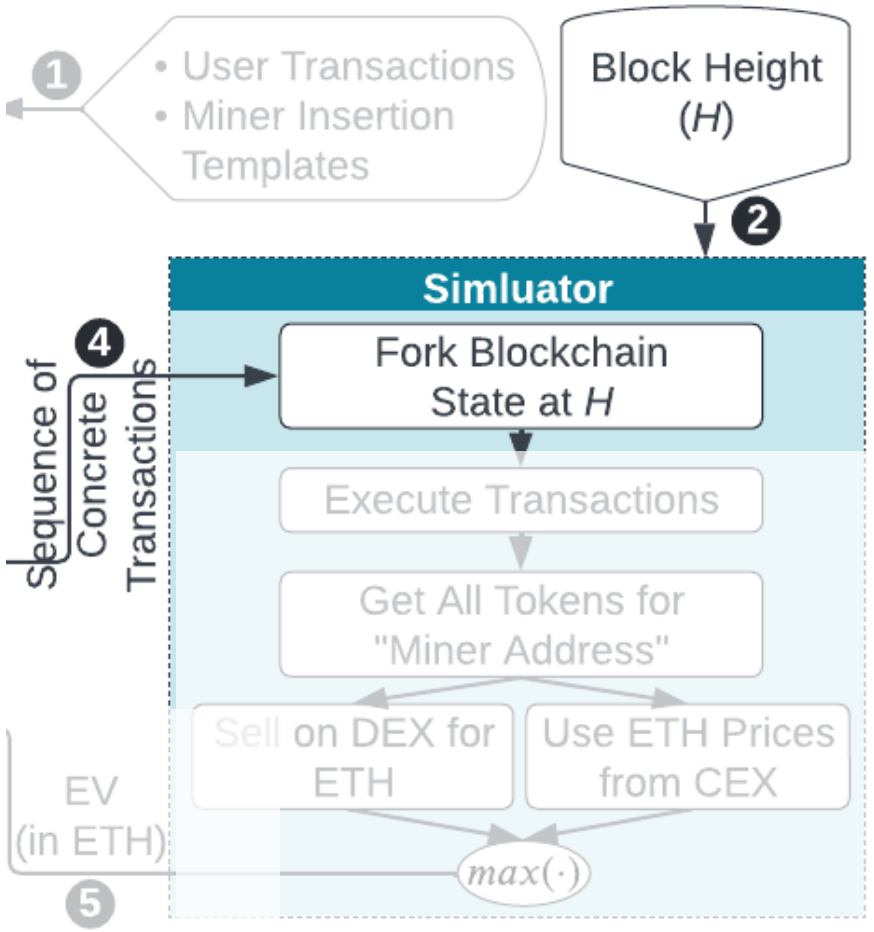
Simulation Module

1. Receive a concrete transaction sequence



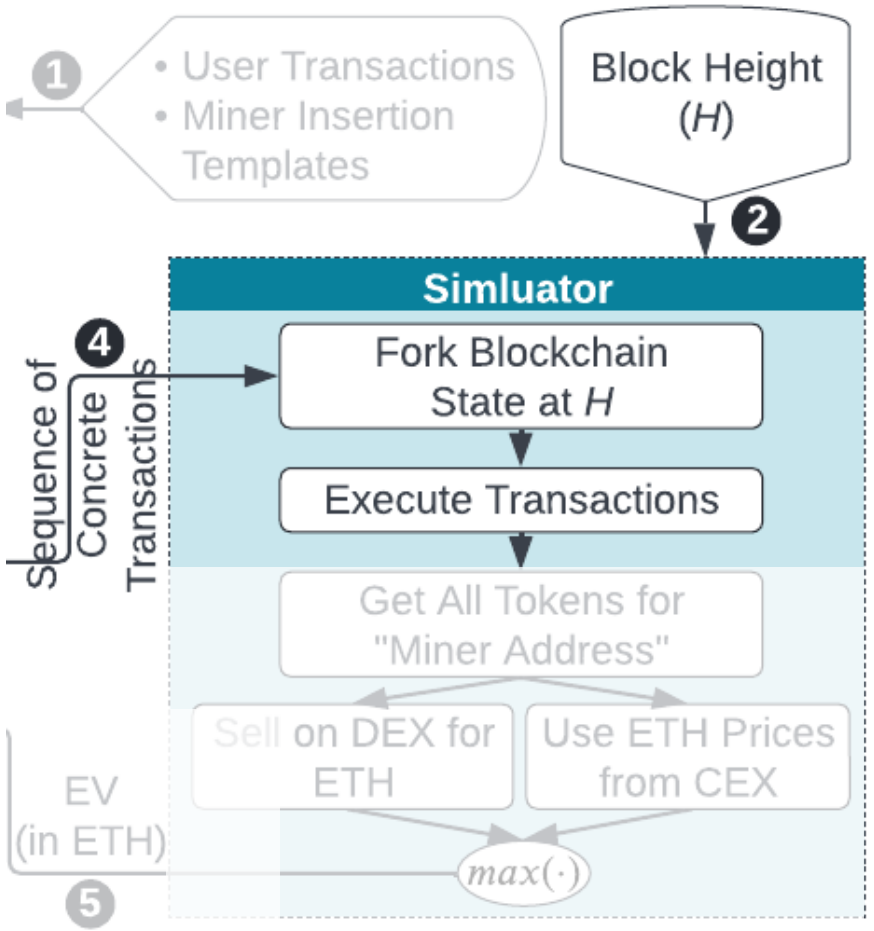
Simulation Module

1. Receive a concrete transaction sequence
2. Fork the blockchain state at height *H*
3. Give miner an initial capital (e.g. 10m Eth)



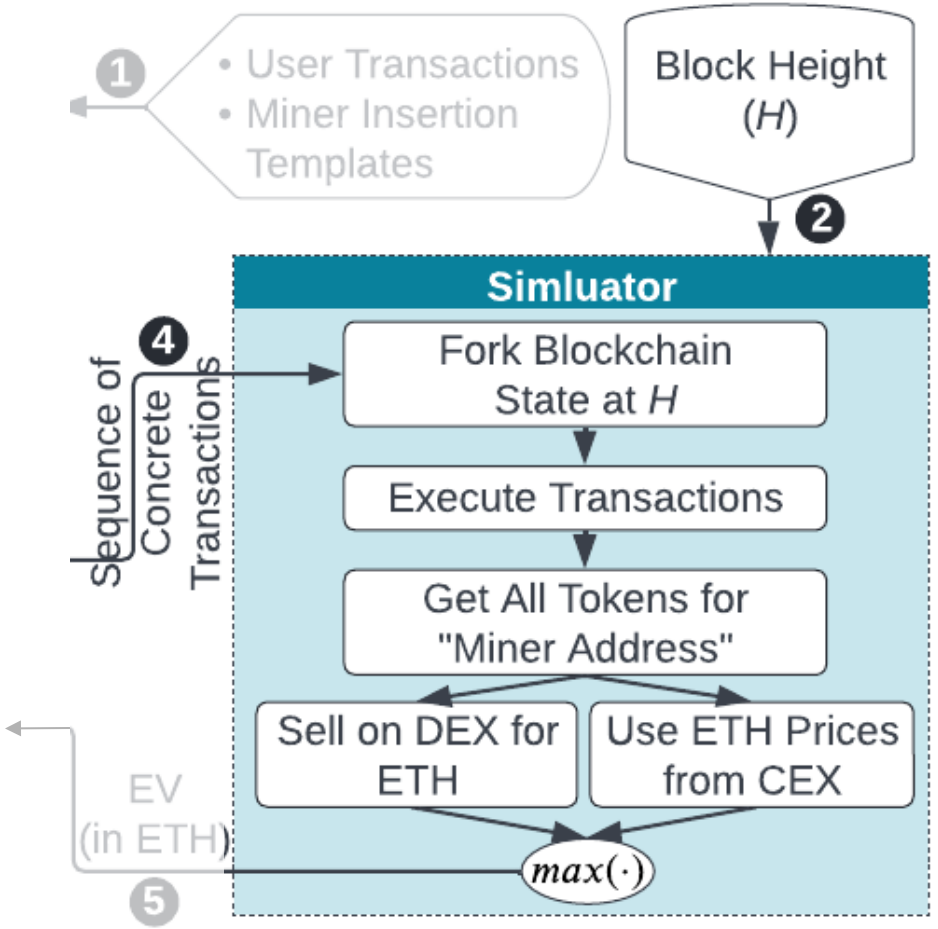
Simulation Module

1. Receive a concrete transaction sequence
2. Fork the blockchain state at height *H*
3. Give miner an initial capital (e.g. 10m Eth)
4. Execute transaction sequence



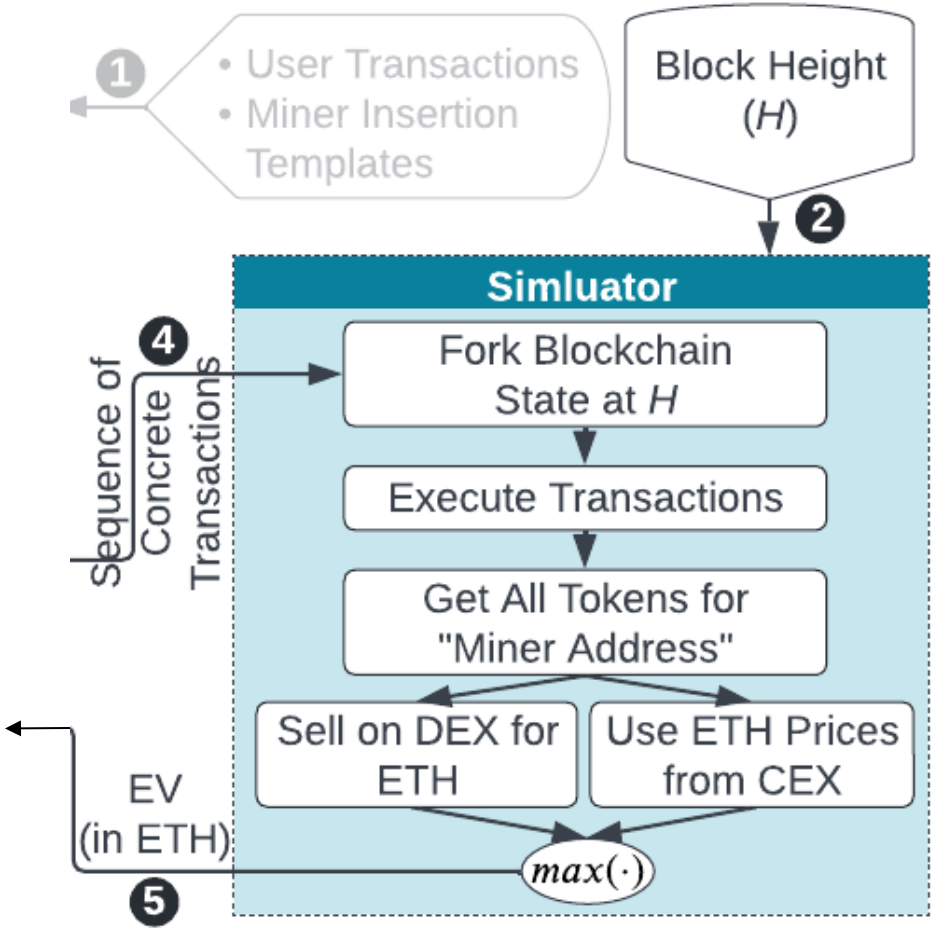
Simulation Module

1. Receive a concrete transaction sequence
2. Fork the blockchain state at height *H*
3. Give miner an initial capital (e.g. 10m Eth)
4. Execute transaction sequence
5. Convert any non-Eth tokens to Eth by executing DEX trades or using historical prices from CEX



Simulation Module

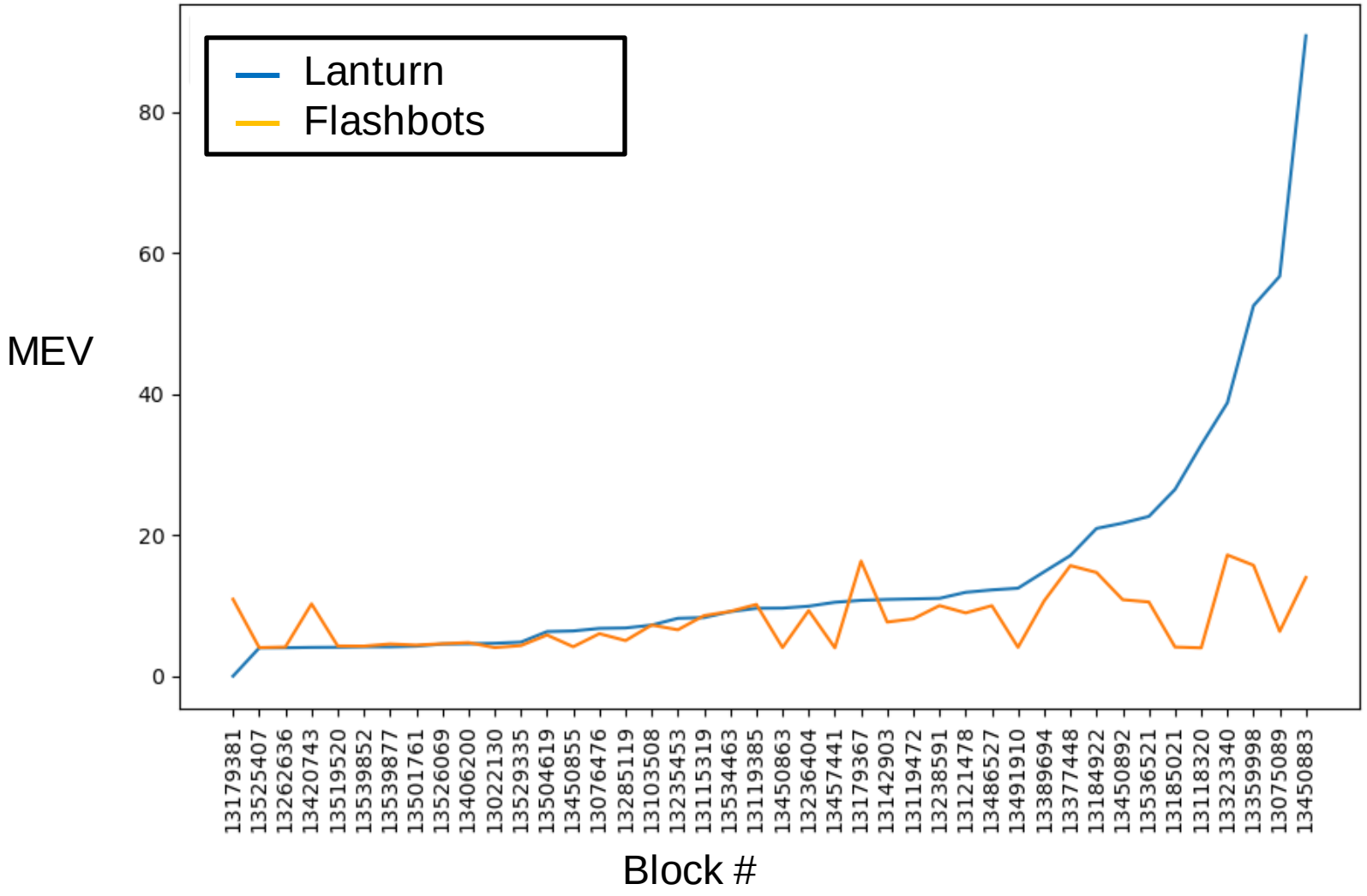
1. Receive a concrete transaction sequence
2. Fork the blockchain state at height *H*
3. Give miner an initial capital (e.g. 10m Eth)
4. Execute transaction sequence
5. Convert any non-Eth tokens to Eth by executing DEX trades or using historical prices from CEX
6. Mine block *H+1*
7. Return the miner's final Eth balance less initial balance.



Evaluation

- Dataset
  - Filter blocks that have more than $1m trade on DEX (Sushiswap)
  - CEX prices: Freely available historical minute-level price data (Binance)
  - Baseline: Flashbots data for MEV bribes paid to the miners, through gas fees and direct transfers
- Template Miner Insertions
- For each pair of tokens traded in user transactions: Insert two symbolic transactions, one for each trading direction

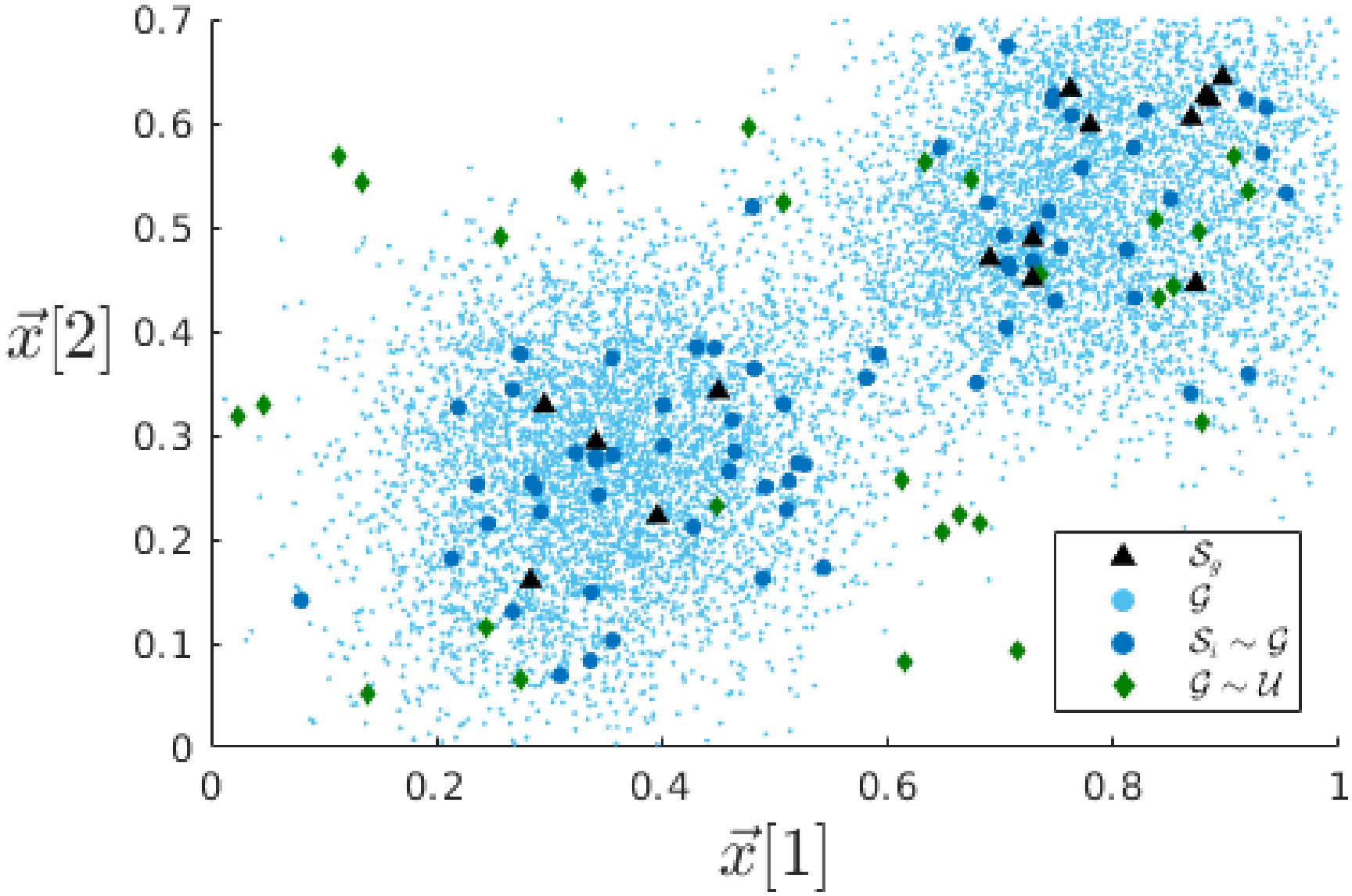
Evaluation


Lanturn
Flashbots
MEV
0
20
40
60
80
13179381
13525407
13262636
13420743
13519520
13539852
13539877
13501761
13526069
13406200
13022130
13529335
13504619
13450855
13076476
13285119
13103508
13235453
13115319
13534463
13119385
13450863
13236404
13457441
13179367
13142903
13119472
13238591
13121478
13486527
13491910
13389694
13377448
13184922
13450892
13536521
13185021
13118320
13323340
13359998
13075089
13450883
Block #

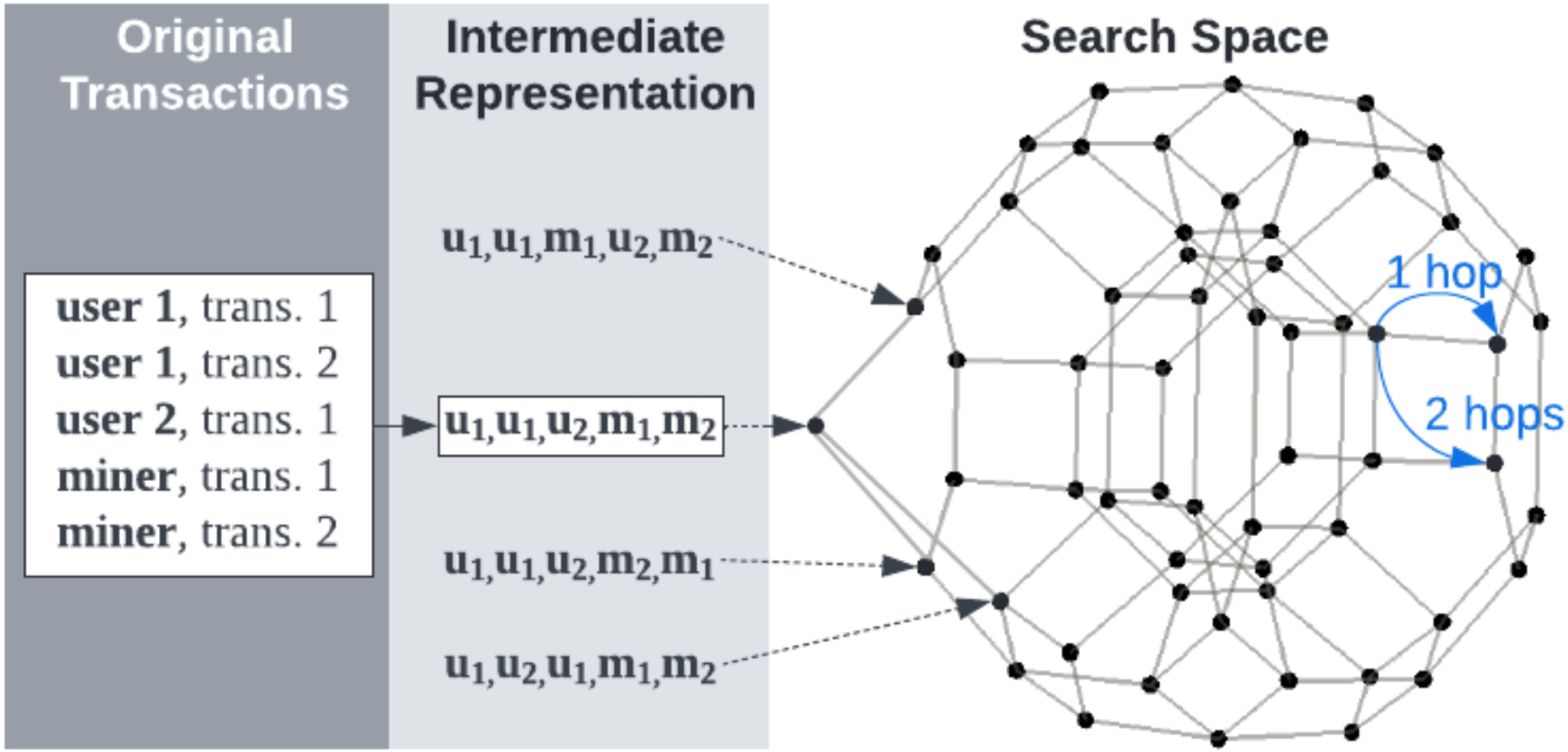
Conclusion

- Formulation of cryptoeconomic smart-contract security as a learning task.
- The approach can generalize to *any* smart-contract by treating smart-contract bytecode as black box simulation environment
- **Lanturn** can scale to MEV strategies spanning over large (>30) number of transactions efficiently, and without modelling (complex) smart-contract behavior.
- **Lanturn** beats the baseline of historical MEV extracted onchain, and uncovers value left on the table by miners/validators.

Mutations


0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
$\vec{x}[2]$
0
0.2
0.4
0.6
0.8
1
$\vec{x}[1]$
S_g
$\mathcal{G}$
$S_i \sim \mathcal{G}$
$\mathcal{G} \sim \mathcal{U}$

Mutations


Original Transactions
Intermediate Representation
Search Space
user 1, trans. 1
user 1, trans. 2
user 2, trans. 1
miner, trans. 1
miner, trans. 2
u_1,u_1,m_1,u_2,m_2
u_1,u_1,u_2,m_1,m_2
u_1,u_1,u_2,m_2,m_1
u_1,u_2,u_1,m_1,m_2
1 hop
2 hops